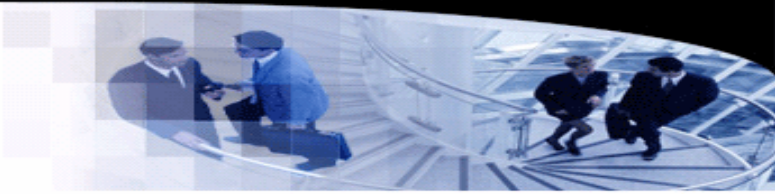
武漢大學
WUHAN UNIVERSITY

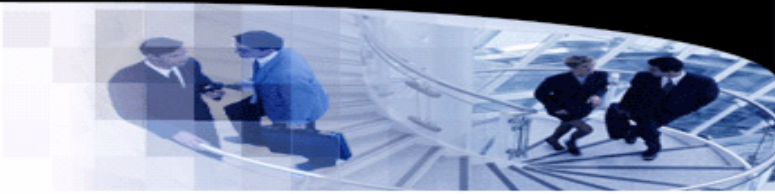


Programming Languages and Compilers

Lecture 13. Parsing

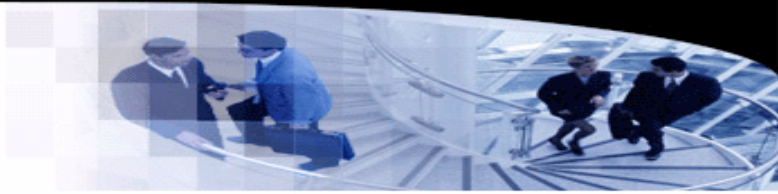
袁梦霆 (YUAN Mengting)
ymt@whu.edu.cn
School of Computer Science, Wuhan University

Scanner and Parser



- Compiler architectures
  - Many compilers use a pipeline-style architecture, which translates a source program into a target program via multiple stages.
  - At each stage, the input representation (of the program) is transformed into the output representation.
- Scanner (or lexer)
  - Scanners are used to transform the source program, which is a string of symbols, into a string of **tokens**.

Scanner and Parser



- Scanner (or lexer)
  - Scanners are used to transform the source program, which is a string of symbols, into a string of **tokens**.

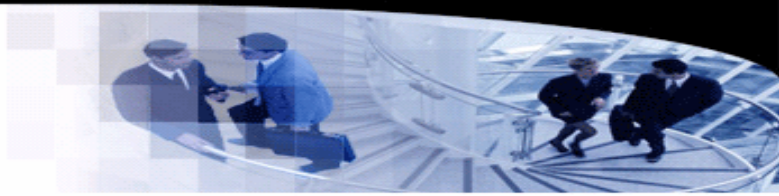
```
ymt@DESKTOP-VPDNSDJ:~$ cat foo.cpp

int bar(double x, int y) {
  int z = x + y++;
  return z;
}
```

Invoke the scanner of clang-14 to translate the source program into a string of tokens.

```
ymt@DESKTOP-VPDNSDJ:~$ clang-14 -fsyntax-only -Xclang -dump-tokens foo.cpp
int 'int'         [StartOfLine]  Loc=<foo.cpp:2:1>
identifier 'bar'          [LeadingSpace] Loc=<foo.cpp:2:5>
l_paren '('               Loc=<foo.cpp:2:8>
double 'double'           Loc=<foo.cpp:2:9>
identifier 'x'    [LeadingSpace] Loc=<foo.cpp:2:16>
comma ','                 Loc=<foo.cpp:2:17>
int 'int'         [LeadingSpace] Loc=<foo.cpp:2:19>
identifier 'y'    [LeadingSpace] Loc=<foo.cpp:2:23>
r_paren ')'               Loc=<foo.cpp:2:24>
l_brace '{'       [LeadingSpace] Loc=<foo.cpp:2:26>
int 'int'         [StartOfLine] [LeadingSpace]   Loc=<foo.cpp:3:3>
identifier 'z'    [LeadingSpace] Loc=<foo.cpp:3:7>
equal '='         [LeadingSpace] Loc=<foo.cpp:3:9>
identifier 'x'    [LeadingSpace] Loc=<foo.cpp:3:11>
plus '+'          [LeadingSpace] Loc=<foo.cpp:3:13>
identifier 'y'    [LeadingSpace] Loc=<foo.cpp:3:15>
plusplus '++'             Loc=<foo.cpp:3:16>
semi ';'                  Loc=<foo.cpp:3:18>
return 'return'   [StartOfLine] [LeadingSpace]   Loc=<foo.cpp:4:3>
identifier 'z'    [LeadingSpace] Loc=<foo.cpp:4:10>
semi ';'                  Loc=<foo.cpp:4:11>
r_brace '}'       [StartOfLine]  Loc=<foo.cpp:5:1>
eof ''           Loc=<foo.cpp:5:2>
```

Scanner and Parser



- Parser
  - Parsers check whether the input string of tokens is well-formed, and translate the input into an **intermediate representation** (an abstract syntax tree, for example).

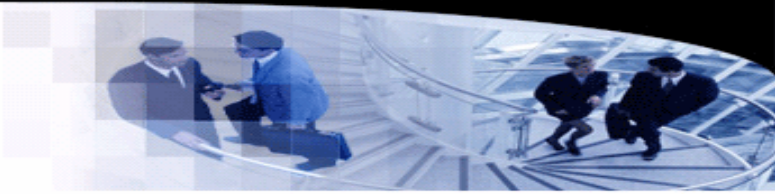
```
ymt@DESKTOP-VPDNSDJ:~$ cat foo.cpp

int bar(double x, int y) {
  int z = x + y++;
  return z;
}
```

```
ymt@DESKTOP-VPDNSDJ:~$ clang-14 -fsyntax-only -Xclang -dump-tokens foo.cpp
int 'int'       [StartOfLine]  Loc=<foo.cpp:2:1>
identifier 'bar'        [LeadingSpace] Loc=<foo.cpp:2:5>
l_paren '('             Loc=<foo.cpp:2:8>
double 'double'         Loc=<foo.cpp:2:9>
identifier 'x'  [LeadingSpace] Loc=<foo.cpp:2:16>
comma ','               Loc=<foo.cpp:2:17>
int 'int'       [LeadingSpace] Loc=<foo.cpp:2:19>
identifier 'y'  [LeadingSpace] Loc=<foo.cpp:2:23>
r_paren ')'             Loc=<foo.cpp:2:24>
l_brace '{'     [LeadingSpace] Loc=<foo.cpp:2:26>
int 'int'       [StartOfLine] [LeadingSpace]   Loc=<foo.cpp:3:3>
identifier 'z'  [LeadingSpace] Loc=<foo.cpp:3:7>
equal '='       [LeadingSpace] Loc=<foo.cpp:3:9>
identifier 'x'  [LeadingSpace] Loc=<foo.cpp:3:11>
plus '+'        [LeadingSpace] Loc=<foo.cpp:3:13>
identifier 'y'  [LeadingSpace] Loc=<foo.cpp:3:15>
plusplus '++'           Loc=<foo.cpp:3:16>
semi ';'                Loc=<foo.cpp:3:18>
return 'return' [StartOfLine] [LeadingSpace]   Loc=<foo.cpp:4:3>
identifier 'z'  [LeadingSpace] Loc=<foo.cpp:4:10>
semi ';'                Loc=<foo.cpp:4:11>
r_brace '}'     [StartOfLine]  Loc=<foo.cpp:5:1>
eof ''          Loc=<foo.cpp:5:2>
```

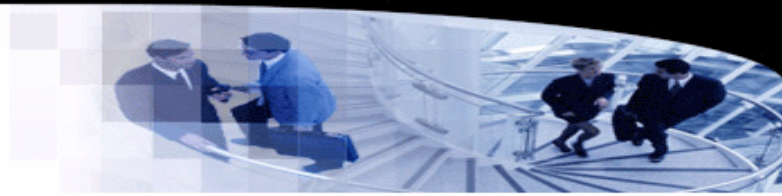
```
ymt@DESKTOP-VPDNSDJ:~$ clang-14 -fsyntax-only -Xclang -ast-dump foo.cpp
TranslationUnitDecl 0x39cac0d8 <<invalid sloc>> <invalid sloc>
|-TypedefDecl 0x39cac940 <<invalid sloc>> <invalid sloc> implicit __int128_t '__int128'
| `-BuiltinType 0x39cac6a0 '__int128'
|-TypedefDecl 0x39cac9b0 <<invalid sloc>> <invalid sloc> implicit __uint128_t 'unsigned __int128'
| `-BuiltinType 0x39cac6c0 'unsigned __int128'
|-TypedefDecl 0x39cacd28 <<invalid sloc>> <invalid sloc> implicit __NSConstantString '__NSConstantString_tag'
| `-RecordType 0x39cacaa0 '__NSConstantString_tag'
|   `-CXXRecord 0x39caca08 '__NSConstantString_tag'
|-TypedefDecl 0x39cacdc0 <<invalid sloc>> <invalid sloc> implicit __builtin_ms_va_list 'char *'
| `-PointerType 0x39cacd80 'char *'
|   `-BuiltinType 0x39cac180 'char'
|-TypedefDecl 0x39cf1798 <<invalid sloc>> <invalid sloc> implicit __builtin_va_list '__va_list_tag[1]'
| `-ConstantArrayType 0x39cf1740 '__va_list_tag[1]' 1
|   `-RecordType 0x39caceb0 '__va_list_tag'
|     `-CXXRecord 0x39cace18 '__va_list_tag'
`-FunctionDecl 0x39cf1960 <foo.cpp:2:1, line:5:1> line:2:5 bar 'int (double, int)'
  |-ParmVarDecl 0x39cf1808 <col:9, col:16> col:16 used x 'double'
  |-ParmVarDecl 0x39cf1888 <col:19, col:23> col:23 used y 'int'
  `-CompoundStmt 0x39cf1bf8 <col:26, line:5:1>
    |-DeclStmt 0x39cf1b98 <line:3:3, col:18>
    | `-VarDecl 0x39cf1a70 <col:3, col:16> col:7 used z 'int' cinit
    |   `-ImplicitCastExpr 0x39cf1b80 <col:11, col:16> 'int' <FloatingToIntegral>
    |     `-BinaryOperator 0x39cf1b60 <col:11, col:16> 'double' '+'
    |       |-ImplicitCastExpr 0x39cf1b30 <col:11> 'double' <LValueToRValue>
    |       | `-DeclRefExpr 0x39cf1ad8 <col:11> 'double' lvalue ParmVar 0x39cf1808 'x' 'double'
    |       `-ImplicitCastExpr 0x39cf1b48 <col:15, col:16> 'double' <IntegralToFloating>
    |         `-UnaryOperator 0x39cf1b18 <col:15, col:16> 'int' postfix '++'
    |           `-DeclRefExpr 0x39cf1af8 <col:15> 'int' lvalue ParmVar 0x39cf1888 'y' 'int'
    `-ReturnStmt 0x39cf1be8 <line:4:3, col:10>
      `-ImplicitCastExpr 0x39cf1bd0 <col:10> 'int' <LValueToRValue>
        `-DeclRefExpr 0x39cf1bb0 <col:10> 'int' lvalue Var 0x39cf1a70 'z' 'int'
```

Scanner



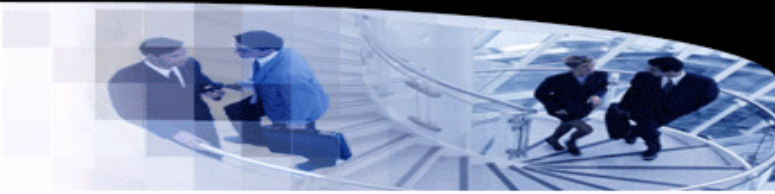
- Generating a scanner
  - Let us revisit the properties of regular languages.
  - In most programming languages, the language of all tokens is a regular language.
  - Each regular language has an equivalent finite automaton.
  - As regular expressions are friendly to programmers, we can define the language in regular expressions.
  - The regular expressions can be converted into a finite automaton by Thompson's construction.
  - A scanner, which simulate the automaton, is then generated.

Scanner



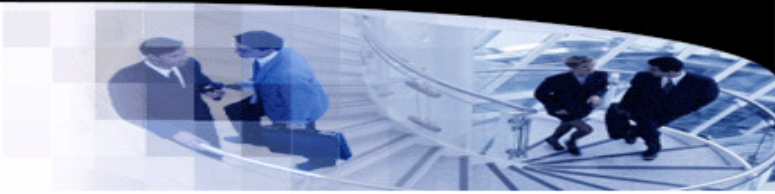
- Generating a scanner
  - There is a family of tools derived from Unix Lex.
  - Gnu flex, or the “fast lexical analyzer”, is an open-source version of Lex.
    - https://www.gnu.org/software/flex/
  - OCaml, however, also provides ocamllex, a lexer generator.
    - https://ocaml.org/manual/5.3/lexyacc.html

Scanner

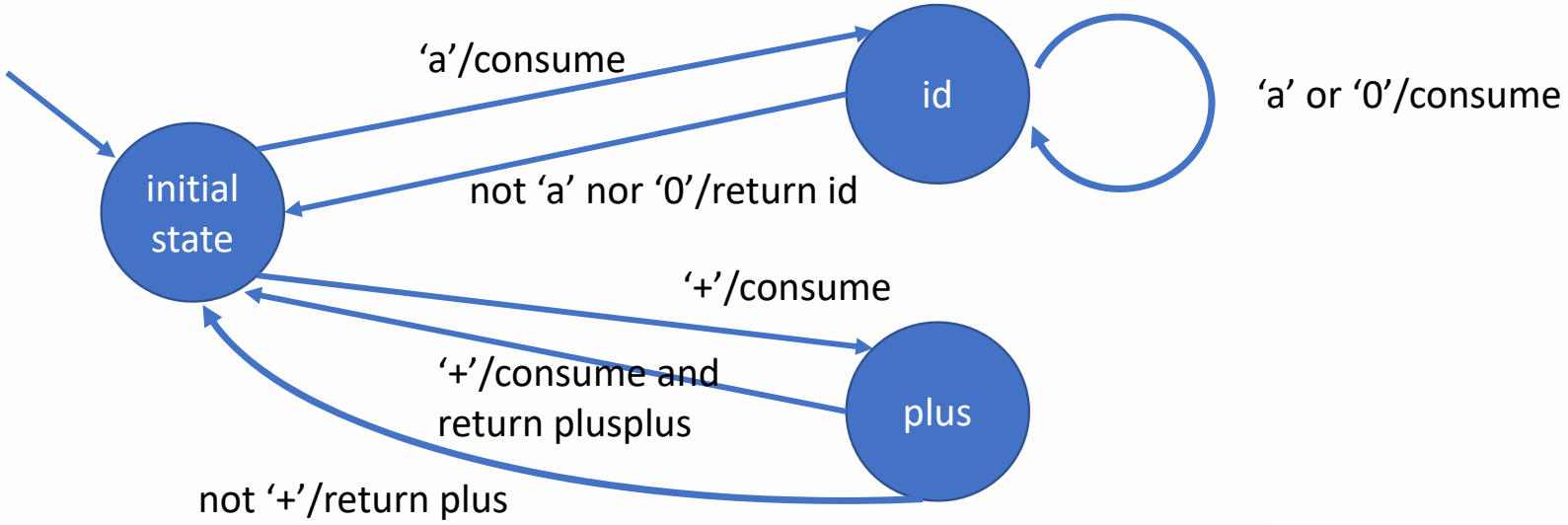


- Hand writing scanner
  - An alternative approach is to write a scanner directly, which also simulates the automaton.
  - However, there are subtle differences between the automaton and the scanner, as the automaton stops when a token is accepted.
  - The scanner recognizes a sequence of tokens, which implies that once a token is recognized, the scanner should return to the initial state and prepares for the next token.
  - A variant of automaton is usually used to handle such issues.
  - Compared to generated scanners, hand writing scanners are more flexible.

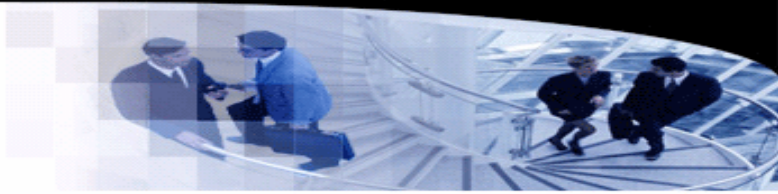
Scanner



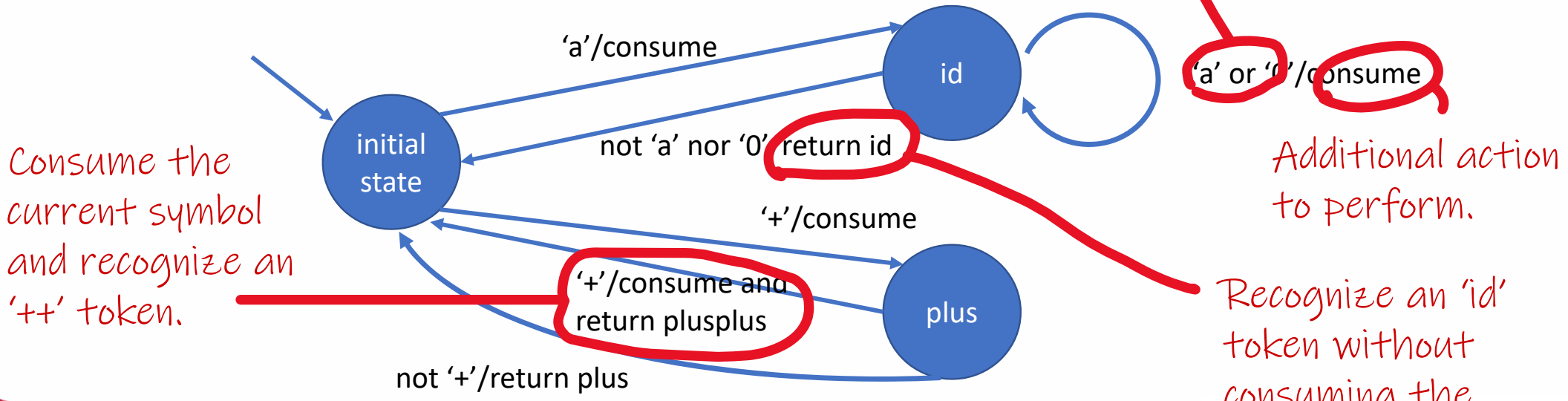
- Hand writing scanner
  - Example:
    - We present an automaton to recognize a sequence of tokens.
    - While the original automaton always consumes the current symbol for each transition, the modified automaton may choose not to consume the current symbol.



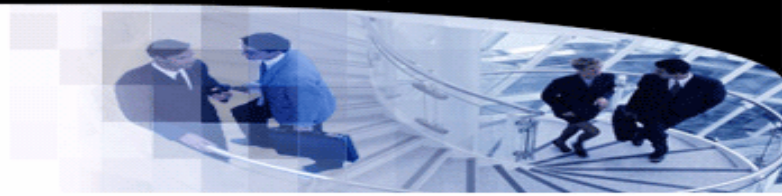
Scanner



- Hand writing scanner
  - Example:
    - We present an automaton to recognize a sequence of tokens.
    - While the original automaton always consumes the current symbol for each transition, the modified automaton may choose not to consume the current symbol.



Scanner



- Hand writing scanner
  - Example:
    - A pure functional implementation (less effective).
    - Note that recursive functions are used to represent states.

```
type token =
  | Identifier
  | Plus
  | PlusPlus
  | Number;;
```

```
string -> string
let tail s = String.sub s 1 ((String.length s) - 1);;
```

```
let rec lex input =
  match String.get input 0 with
  | exception (Invalid_argument ia) -> []
  | '+' -> let t = lex_plus (tail input) in
      (fst t) :: lex (snd t)
  | 'a' -> let t = lex_id "a" (tail input) in
      (fst t) :: lex (snd t)
  | ' ' -> lex (tail input)
  | _ -> raise (Failure "Error")
;;
```

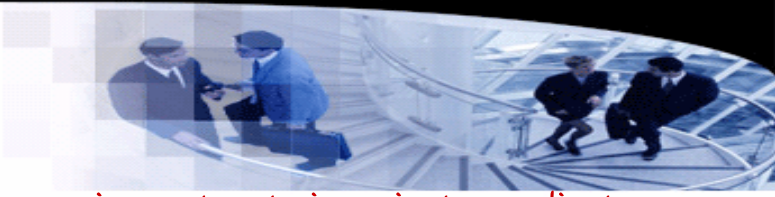
a

```
let rec lex_id acc input =
  match String.get input 0 with
  | 'a' | '0' -> lex_id (acc ^ "a") (tail input)
  | exception (Invalid_argument ia) -> (Identifier, "")
  | _ -> (Identifier, input);;
```

+

```
let lex_plus input =
  match String.get input 0 with
  | '+' -> (PlusPlus, tail input)
  | exception (Invalid_argument ia) -> (Plus, input)
  | _  -> (Plus, input);;
```

Scanner



- Hand writing scanner
  - Example:
    - A pure functional implementation (less effective).
    - Note that recursive functions are used to represent states.

Translate an input string into a list of tokens.

Returns a token and the remaining input string.

```
type token =
  | Identifier
  | Plus
  | PlusPlus
  | Number;;
```

```
string -> string
let tail s = String.sub s 1 ((String.length s) - 1);;
```

```
let rec lex input =
  match String.get input 0 with
  | exception (Invalid_argument ia) -> []
  | '+' -> let t = lex_plus (tail input) in
      (fst t) :: lex (snd t)
  | 'a' -> let t = lex_id "a" (tail input) in
      (fst t) :: lex (snd t)
  | ' ' -> lex (tail input)
  | _ -> raise (Failure "Error")
;;
```

a

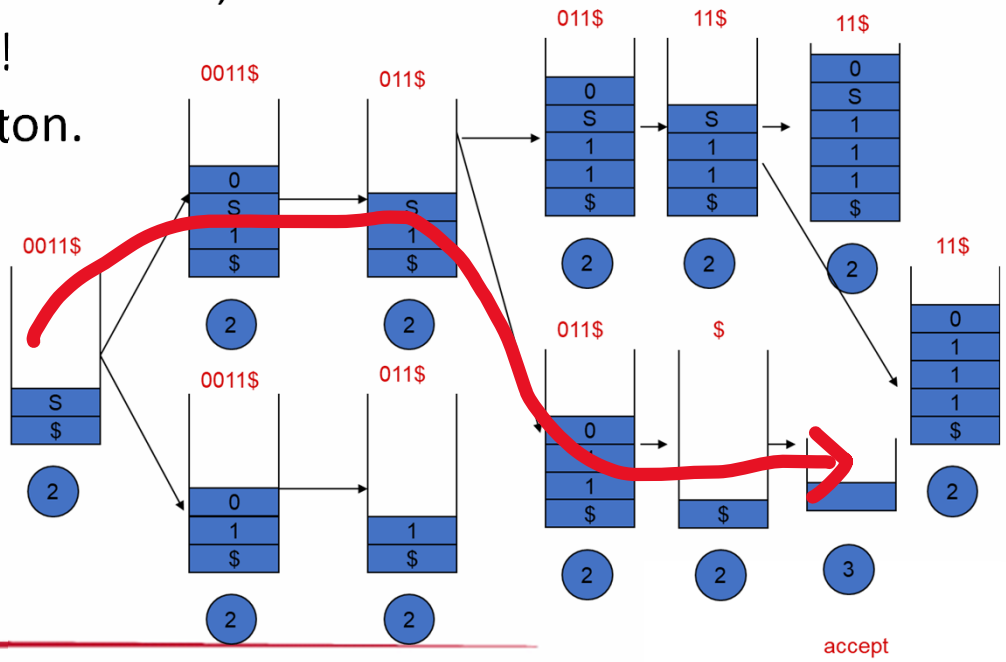
```
let rec lex_id acc input =
  match String.get input 0 with
  | 'a' | '0' -> lex_id (acc ^ "a") (tail input)
  | exception (Invalid_argument ia) -> (Identifier, "")
  | _ -> (Identifier, input);;
```

+

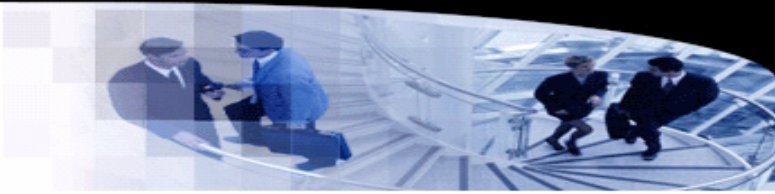
```
let lex_plus input =
  match String.get input 0 with
  | '+' -> (PlusPlus, tail input)
  | exception (Invalid_argument ia) -> (Plus, input)
  | _ -> (Plus, input);;
```

Parser

- Pushdown automata
  - Given a grammar G and an input α, a parser of G checks whether $\alpha \in \mathcal{L}(G)$.
  - In lecture 6, we know that there exists an equivalent pushdown automaton that recognizes $\mathcal{L}(G)$.
  - However, the automaton is nondeterministic, which is not tractable.
  - If only we knew the "correct" trace!
  - A **deterministic** pushdown automaton.

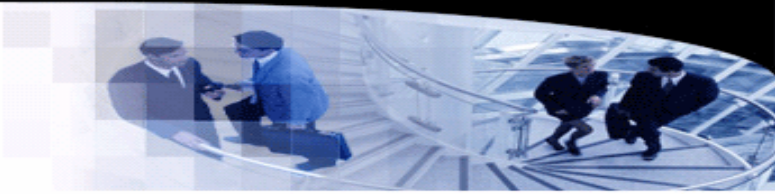


Parser



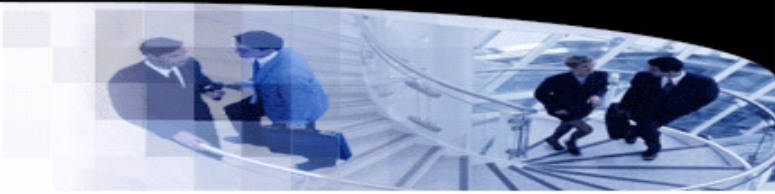
- Recursive descent parser
  - Consider the production $A \rightarrow aB$ and the input α.
  - When A is in the stack, suppose the pushdown automaton selects aB to replace A and tries to match aB with α.
  - **Observation**: α should start with a. Otherwise the match fails. Once a is matched, the automaton matches B with the remaining input string **recursively**.
  - Now let us consider $A \rightarrow \gamma_1|\gamma_2|\ldots|\gamma_n$. When the input $\alpha = a\beta$, which production should we select to replace A in the stack?
  - For each γ_i, let $First(\gamma_i) = \{x|\gamma_i \Rightarrow^* x\delta\}$.
  - **Conclusion**: If there is a γ_i such that $a \in First(\gamma_i)$, then γ_i is selected.
  - The intuition is that γ_i seems to have the potential to match α based on the **look ahead symbol** a.
  - The calculation of $First$ will be introduced in the next lecture. For now, we just calculate $Fisrt$ with our naked eyes.

Parser



- Recursive descent parser
  - Given a grammar $G = (V_N, V_T, P, S)$, we design a set of recursive functions for each nonterminal of V_N.
  - If $A \in V_N$, we design the function `parse_a` to match a string δ, where $A \Rightarrow^* \delta$, in the input string α.
  - Inside `parse_a`, the recursive function handles all productions of A case by case.
  - For each production $A \rightarrow \gamma_i = b_1 b_2 \ldots b_m$, parse_a checks the symbols at the right-hand side of the production:
    - If b_j is a terminal, match b_j with the first symbol of the input.
    - If b_j is a nonterminal, invoke `parse_b`$_j$ recursively.
  - When `parse_s` returns and the input is empty, we know `parse_s` has matched the input successfully.

Parser

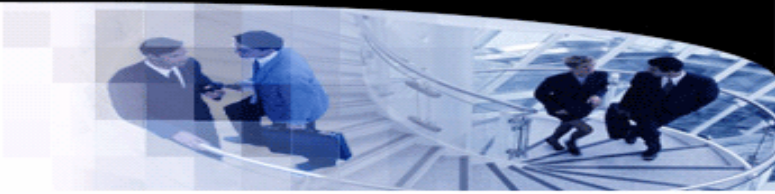


- Recursive descent parser
  - Example: Consider the following grammar $G[E]$. Write a recursive descent parser.
    - $E \rightarrow TE'$
    - $E' \rightarrow +TE'|\epsilon$
    - $T \rightarrow FT'$
    - $T' \rightarrow * FT'|\epsilon$
    - $F \rightarrow (E)|i$
  - We demonstrate how to write recursive functions (i.e., `e`, `e1`, `t`, `t1`, and `f`) for the nonterminals.
  - Suppose the input is a string. We convert the string into a list of characters and parse the list.

```
string -> char list
let parse input =
  let explode s = List.init (String.length s) (String.get s) in
    e (explode input);;
```

  - Here, function `e` is supposed to match all well-formed expressions in the input.

Parser

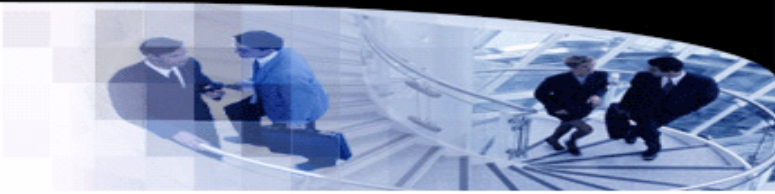


- Recursive descent parser
  - Consider the nonterminal E.
    - $E \rightarrow TE'$

```
(* e -> te' *)
char list -> char list
let rec e s =
  match s with
  | '('::ts | 'i'::ts -> e1 (t s)
  | _ -> raise (Failure "Error e")
```

  - Each recursive parsing function tries to match a string in the input s, and returns the remaining string (after the match).
  - An expression should start with '(' or 'I'.
    - In this case, we invoke t to match a string. Then `e1` is invoked to match another string in the remaining string returned by `t`.
  - Other symbols cannot be matched.

Parser

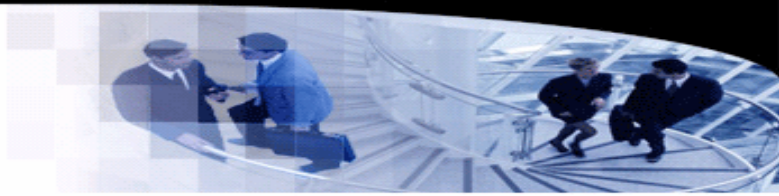


- Recursive descent parser
  - Consider the nonterminal E'.
    - $E' \rightarrow +TE' | \epsilon$

```
(* e' -> +te' | epsilon *)
char list -> char list
and e1 s =
  match s with
  | '+'::ts -> e1 (t ts)
  | '$'::ts | ')'::ts -> s
  | _ -> raise (Failure "Error e1")
```

  - Now we have two productions.
  - The first production starts with '+'. Hence, the function matches '+', takes the remaining string `ts`, and passes `ts` to "`e1 t`" which matches TE'.
  - As for the second production, the function does nothing but returns the current input. The intuition is that $E \rightarrow \epsilon$ matches nothing in the input.
    - Note that '$' and ')' are calculated by a $Follow$ function, which will be introduced in the next lecture.

Parser

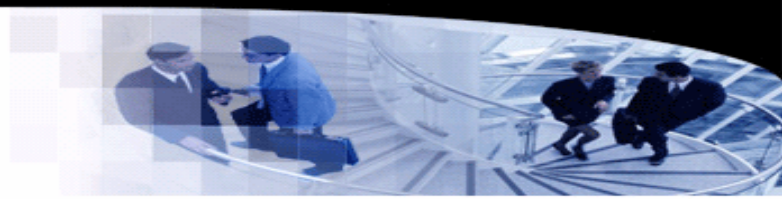


- Recursive descent parser
  - Consider the nonterminal F.
    - $F \rightarrow (E)|i$

```
(* f -> (e) | i *)
char list -> char list
and f s =
  match s with
  | '('::ts ->
      (match (e ts) with
      | ')'::tt -> tt
      | _ -> raise (Failure "Error f"))
  | 'i'::ts -> ts
  | _ -> raise (Failure "Error f")
  ;;
```

  - For the first production, it matches a '(', invokes e, and matches a ')'.
  - For the second production, it simply matches an 'i'.

Parser



- Recursive descent parser
  - The whole program for the example.

```
(* e -> te' *)
char list -> char list
let rec e s =
  match s with
  | '('::ts | 'i'::ts -> e1 (t s)
  | _ -> raise (Failure "Error e")

(* e' -> +te' | epsilon *)
char list -> char list
and e1 s =
  match s with
  | '+'::ts -> e1 (t ts)
  | '$'::ts | ')'::ts -> s
  | _ -> raise (Failure "Error e1")

(* t -> ft' *)
char list -> char list
and t s =
  match s with
  | '('::ts | 'i'::ts -> t1 (f s)
  | _ -> raise (Failure "Error t")
```

```
(* t1 -> *ft1 | epsilon *)
char list -> char list
and t1 s =
  match s with
  | '*'::ts -> t1 (f ts)
  | '$'::ts | ')'::ts | '+'::ts -> s
  | _ -> raise (Failure "Error t1")

(* f -> (e) | i *)
char list -> char list
and f s =
  match s with
  | '('::ts ->
      (match (e ts) with
      | ')'::tt -> tt
      | _ -> raise (Failure "Error f"))
  | 'i'::ts -> ts
  | _ -> raise (Failure "Error f")
  ;;


string -> char list
let parse input =
  let explode s = List.init (String.length s) (String.get s) in
    e (explode input);;
```